NATIONAL INSTITUTE OF TECHNOLOGY SIKKIM

Department of Computer Science & Engineering

Classification of Human-Written and AI-Generated Text

Using Machine Learning and Fine-Tuned Transformer Models

Submitted by

Pravesh Gedon Sharma (B240019)

Ankit Rawat (B320032)

2026

Table of Contents

Abstract 3

1. Problem Statement 3

2. Objectives 3

3. Proposed Architecture 3

4. Phase 1 — Traditional Machine Learning 5

4.1 Text Preprocessing 5

4.2 Linguistic and Stylometric Feature Extraction 5

4.3 Feature Extraction — TF-IDF 8

4.4 Semantic Feature Extraction 9

4.5 Classification Models 10

5. Phase 2 — Fine-Tuned Open-Source Language Model 10

5.1 Model Selection 10

5.2 Tokenization and Input Preparation 10

5.3 Transformer Architecture 10

5.4 Fine-Tuning 11

6. Phase 3 — Experimental Evaluation 11

6.1 Experimental Setup 11

6.2 Ablation Study 12

6.3 Performance Evaluation 12

7. Technology Stack 13

8. Streamlit Demonstration (Concept) 13

9. Overall Project Flow 13

Abstract

The proliferation of large language models such as ChatGPT has made it increasingly difficult to distinguish human-authored text from machine-generated text. This project presents a three-phase approach to this classification problem. Phase 1 is a traditional machine learning pipeline that combines stylometric and linguistic features, TF-IDF word and character n-grams, and semantic sentence-embedding features, classified with Logistic Regression, Linear SVM, Random Forest, and optional feature-fusion models. Phase 2 fine-tunes open-source encoder-based Transformers (DistilBERT and RoBERTa) for sequence classification. Phase 3 evaluates and compares the approaches using Accuracy, Precision, Recall, F1-score, ROC-AUC, and confusion matrices, supported by an ablation study of lexical, stylistic, semantic, and hybrid features and by a generalization test on AI generators unseen during training. A Streamlit-based demonstration application is proposed as the final deliverable.

1. Problem Statement

With the increasing use of ChatGPT and other generative AI systems, it is useful to automatically classify whether a given piece of text was written by a human or generated by an AI model. This project compares a traditional NLP/ML approach, enriched with stylometric and semantic features, against a fine-tuned open-source language model on this task.

2. Objectives

- Use a labeled dataset with 0 = human-written and 1 = AI-generated text.
- Preprocess and analyze the text through exploratory data analysis (EDA).
- Extract linguistic, stylometric, TF-IDF, and semantic features for a traditional machine learning classifier.
- Compare Logistic Regression, Linear SVM, Random Forest, and feature-fusion models.
- Fine-tune an open-source Transformer model such as DistilBERT or RoBERTa.
- Run an ablation study to measure the contribution of lexical, stylistic, semantic, and hybrid features.
- Compare all approaches using Accuracy, Precision, Recall, F1-score, ROC-AUC, and a confusion matrix.
- Analyze the advantages and limitations of automated AI-text detection.

3. Proposed Architecture

Figure 1 summarizes the end-to-end pipeline. Both modelling branches are trained independently on an identical data split, then compared side by side during evaluation.

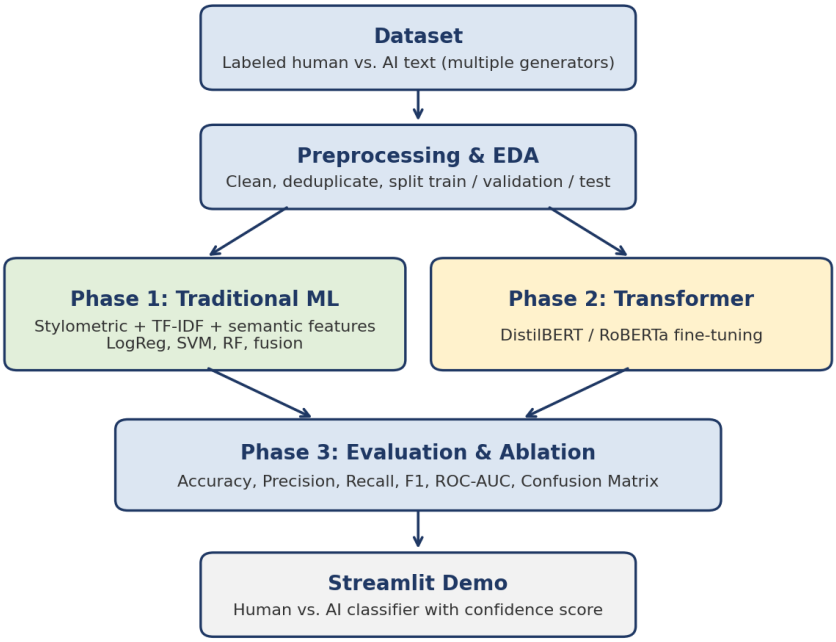


Figure 1. End-to-end system pipeline.

4. Phase 1 — Traditional Machine Learning

This chapter presents the final methodology. Phase 1 builds an interpretable feature-based baseline; Phase 2 fine-tunes a Transformer; Phase 3 evaluates both under a common protocol. Figure 2 shows the detailed Phase 1 pipeline: after preprocessing and a stratified split, three feature families (stylometric, TF-IDF, and semantic) are extracted in parallel, scaled, fused, and passed to the classifiers.

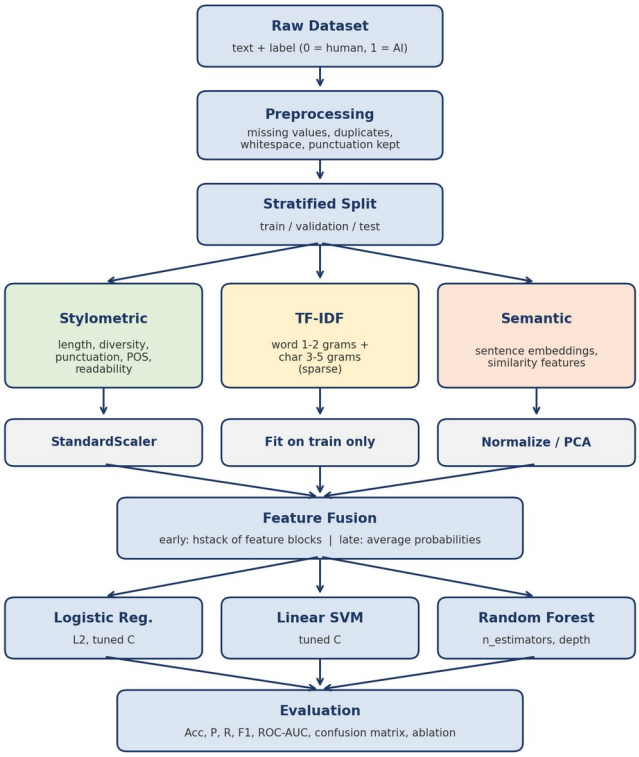


Figure 2. Detailed pipeline of the traditional machine learning branch (Phase 1).

4.1 Text Preprocessing

- **Missing values:** rows with empty or null text or labels are removed.
- **Duplicate removal:** exact and near-duplicate samples are dropped so that no text appears in both the training and test sets.
- **Whitespace normalization:** repeated spaces, tabs, and line breaks are collapsed into single spaces.
- **Punctuation preservation:** punctuation and writing style carry useful signal, so aggressive stripping is avoided.
- **Text normalization:** lowercasing (for TF-IDF only), Unicode normalization, and optional stop-word removal. Stylometric features are computed on the raw, un-lowercased text.

4.2 Linguistic and Stylometric Feature Extraction

Stylometry describes how a text is written rather than what it says. Human authors vary sentence length, vocabulary, and punctuation irregularly, whereas language models tend to produce smoother, more uniform text. Each document is converted into a dense vector of roughly 40 to 60 numeric

features in five groups (Figure 3). All features are computed on the raw text and standardized with StandardScaler, fitted on the training set only.

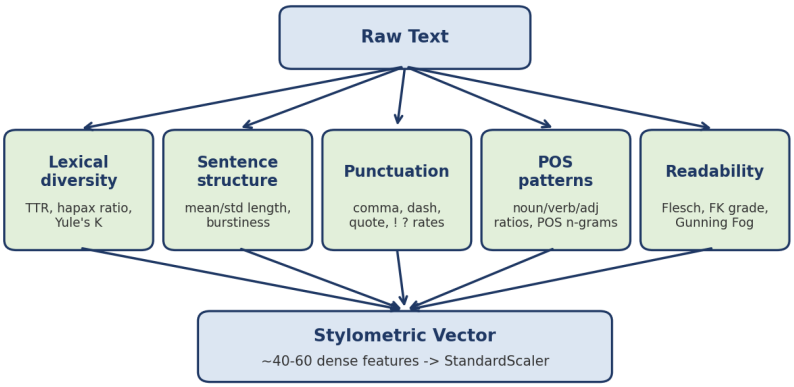


Figure 3. Stylometric feature groups extracted from each document.

Let N be the number of word tokens, V the number of distinct words, and S the number of sentences.

Group	Features	Definition / rationale
Lexical diversity	Type-token ratio (TTR); hapax ratio; average word length	In simple words: how varied the vocabulary is. TTR = different words / total words (V / N). Example: in "the cat saw the cat" there are 3 different words out of 5, so TTR = 0.6. A higher value means fewer repeated words. Hapax ratio = words used only once / N. TTR drops as texts get longer, so texts are compared over equal-length windows.
Sentence structure	Mean, standard deviation, min and max sentence length; burstiness; words per paragraph	Burstiness = σ / μ of sentence lengths. Humans mix short and long sentences, so burstiness tends to be higher than for AI text.
Punctuation	Rates of commas, semicolons, colons, dashes, quotation marks, parentheses, ! and ? per 100 characters; capital-letter ratio	Punctuation habits are strong style markers (for example, em-dash and comma frequency differ between authors and models).
POS patterns	Ratios of nouns, verbs, adjectives, adverbs, pronouns, determinants; function-word frequency; top POS bigrams	Tags come from spaCy or NLTK. Syntactic habits are less topic-dependent than word choice, which helps generalization across domains.
Readability	Flesch Reading Ease; Flesch-Kincaid Grade; Gunning Fog; average syllables per word	How easy the text is to read, based on sentence length and word length. Explained step by step below.

Readability indices. These are simple scores that estimate how easy a text is to read. All three use only two ideas: how long the sentences are and how long or complex the words are.

Symbol	Meaning
N	Total number of words in the text
S	Total number of sentences in the text
N / S	Average sentence length (words per sentence). Longer sentences are harder to read.
syllables / N	Average syllables per word. Words with more syllables are harder to read ("cat" = 1, "beautiful" = 3).
Complex words	Words with 3 or more syllables (e.g. "important", "information")

1. Flesch Reading Ease (FRE). The score starts at 206.835, then subtracts a penalty for long sentences (1.015 per word of average sentence length) and a bigger penalty for long words (84.6 per syllable of average word length). Higher score = easier text.

$$\textit{Flesch Reading Ease} = 206.835 - 1.015 \times (N / S) - 84.6 \times (\textit{syllables} / N)$$

FRE score	How the text reads
90 – 100	Very easy (primary school level)
60 – 70	Plain English, easily understood by most adults
30 – 50	Difficult (college level)
0 – 30	Very difficult (academic or legal writing)

2. Flesch-Kincaid Grade Level (FKGL). Uses the same two ingredients but gives the result as a school grade. A score of 9 means a 9th-grade student can understand the text. Here a higher score means harder text, the opposite of FRE.

$$\textit{Flesch-Kincaid Grade} = 0.39 \times (N / S) + 11.8 \times (\textit{syllables} / N) - 15.59$$

3. Gunning Fog Index. Instead of counting syllables for every word, it counts only the complex words (3 or more syllables) and gives the years of schooling needed to understand the text on first reading. The factor 0.4 scales the result to a grade level.

$$\textit{Gunning Fog} = 0.4 \times [\ (N / S) + 100 \times (\textit{complex words} / N)\ ]$$

Worked example. Suppose a passage has N = 100 words, S = 5 sentences, 150 syllables in total, and 10 complex words. Then N / S = 20 words per sentence and syllables / N = 1.5 per word.

Index	Calculation	Result
Flesch Reading Ease	206.835 − 1.015 × 20 − 84.6 × 1.5	≈ 59.6 (fairly difficult)
Flesch-Kincaid Grade	0.39 × 20 + 11.8 × 1.5 − 15.59	≈ 9.9 (grade 10)
Gunning Fog	0.4 × (20 + 100 × 10 / 100)	12.0

Why it helps: AI-generated text often stays in a narrow, steady readability range, while human text varies more from passage to passage. The three scores are added to the stylometric vector as ordinary numeric features.

Example extraction routine:

```
def stylometric_features(text):
    doc   = nlp(text)                        # spaCy
    sents = [len(s) for s in doc.sents]
    words = [t.text.lower() for t in doc if t.is_alpha]
    feats = {
      'ttr': len(set(words)) / len(words),
      'sent_mean': np.mean(sents), 'sent_std': np.std(sents),
      'burstiness': np.std(sents) / np.mean(sents),
      'comma_rate': 100 * text.count(',') / len(text),
      'flesch': textstat.flesch_reading_ease(text),
    }
    for pos in ['NOUN','VERB','ADJ','ADV','PRON']:
        feats[pos] = sum(t.pos_ == pos for t in doc) / len(doc)
    return feats
```

Because these features are low-dimensional and interpretable, they support feature-importance analysis (for example, Random Forest importances or Logistic Regression coefficients) that shows which stylistic cues separate human from AI text.

4.3 Feature Extraction — TF-IDF

TF-IDF converts each document into a sparse vector that weights a term highly when it is frequent in that document but rare across the corpus (Figure 4).

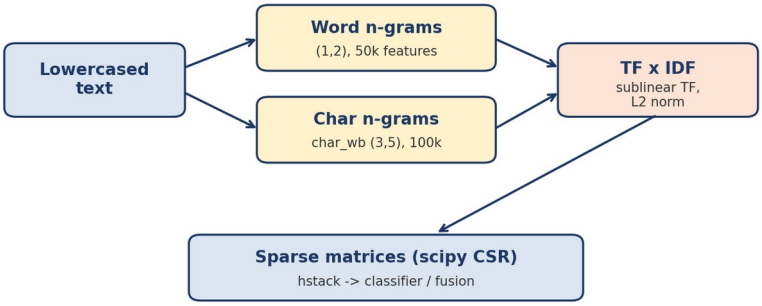


Figure 4. TF-IDF feature extraction with word and character n-grams.

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

$$IDF(t) = \ln[\ (1 + n) / (1 + df(t))\ ] + 1 \qquad \text{(smoothed, as in scikit-learn)}$$

$$\text{sublinear TF:}\ \ TF(t, d) = 1 + \ln(\ count(t, d)\ )$$

Here n is the number of training documents and df(t) is the number of documents containing term t. Each row is then L2-normalized so long and short documents are comparable.

- **Word n-grams (1 to 2):** unigrams capture vocabulary preferences, and bigrams capture common phrasing and transitions (for example, stock connectives that language models overuse). Terms appearing in fewer than 2 documents are dropped (min_df = 2) and the vocabulary is capped at 50,000 features.
- **Character n-grams (3 to 5):** computed with the char_wb analyzer inside word boundaries. They capture sub-word style, spelling, morphology, and punctuation habits, need no tokenizer, and are robust to small edits and paraphrasing.

- **Sparse representation:** both matrices are stored as SciPy CSR arrays, so memory grows with non-zero entries only. They can be concatenated (scipy.sparse.hstack) with the scaled stylometric block for feature fusion.
- **Leakage control:** vectorizers are fitted on the training split only, then applied to validation and test data. Optional chi-square selection can reduce the feature space further.

Suggested vectorizer configuration:

```
word_tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=50000,
                             min_df=2, sublinear_tf=True)
char_tfidf = TfidfVectorizer(analyzer='char_wb', ngram_range=(3, 5),
                             max_features=100000, min_df=3,
sublinear_tf=True)
X_tfidf = hstack([word_tfidf.fit_transform(train_text),
                  char_tfidf.fit_transform(train_text)])
```

	Word n-grams	Character n-grams
Captures	Vocabulary, phrases, topic	Spelling, morphology, punctuation, style
Robustness	Sensitive to paraphrasing	More robust to small edits
Size	Up to 50,000 features	Up to 100,000 features
Cost	Low	Higher (many n-grams)

4.4 Semantic Feature Extraction

Semantic features describe meaning and coherence rather than surface form. They complement stylometry and TF-IDF by capturing how ideas progress through a passage (Figure 5).

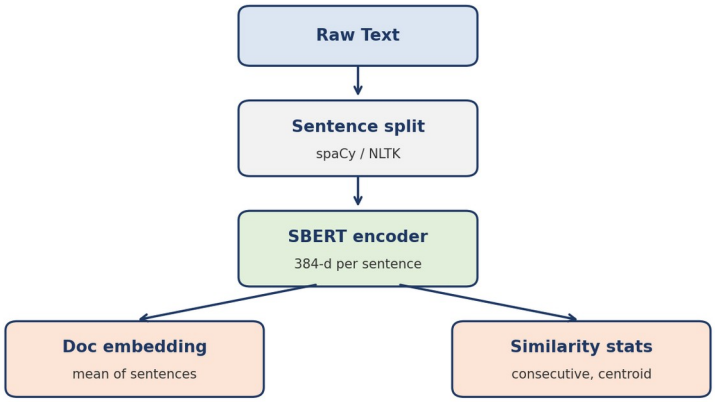


Figure 5. Semantic feature extraction: sentence embeddings and similarity features.

- **Sentence embeddings:** each sentence is encoded with a pretrained Sentence-BERT model such as all-MiniLM-L6-v2 (384 dimensions, mean-pooled token outputs). The document embedding is the mean of its sentence embeddings, giving a dense vector that can be fed to any classifier.

- **Similarity-based features:** let e_i be the embedding of sentence i and c the document centroid. The features are the mean, standard deviation, minimum and maximum of cos(e_i, e_i+1) between consecutive sentences, and of cos(e_i, c) between each sentence and the centroid. High and uniform similarity indicates a very smooth, repetitive flow, which is more typical of AI text; human writing tends to drift and jump more.

```
sbert = SentenceTransformer('all-MiniLM-L6-v2')
E = sbert.encode(sentences, normalize_embeddings=True)   # (S, 384)
consec = (E[:-1] * E[1:]).sum(1)                         # cosine sims
cent   = E @ (E.mean(0) / np.linalg.norm(E.mean(0)))
sem = [E.mean(0), consec.mean(), consec.std(), consec.min(),
       cent.mean(), cent.std()]
```

The 384-dimensional embedding can be used directly or reduced with PCA. The small set of similarity statistics can also be appended to the stylometric block to form a compact dense feature set.

4.5 Classification Models

Logistic Regression and Linear SVM serve as the primary baselines, since sparse TF-IDF features generally perform well with linear classifiers. Random Forest is included as a non-linear point of comparison, particularly for the dense stylometric and semantic features. Optional feature-fusion models concatenate the stylometric, TF-IDF, and semantic feature blocks into one vector (early fusion) or average the probabilities of separately trained models (late fusion). Hyperparameters are tuned by cross-validation on the training split only.

5. Phase 2 — Fine-Tuned Open-Source Language Model

5.1 Model Selection

An encoder-based Transformer suitable for sequence classification is selected. DistilBERT is the practical starting point for a student project, being considerably lighter than larger models, and RoBERTa is used as a stronger second candidate. Encoder models are preferred over generative decoders here because classification needs a bidirectional representation of the whole passage.

5.2 Tokenization and Input Preparation

- **Tokenization:** the model's own subword tokenizer converts raw text into input IDs and an attention mask.
- **Truncation and padding:** sequences are truncated to a maximum length (for example 256 or 512 tokens) and padded dynamically per batch.
- **Document-length handling:** longer documents are split into overlapping chunks (sliding window); chunk probabilities are averaged to give one document-level prediction.

5.3 Transformer Architecture

Transformer pipeline: raw text → tokenizer → input IDs and attention mask → pretrained encoder → classification head → human / AI label.

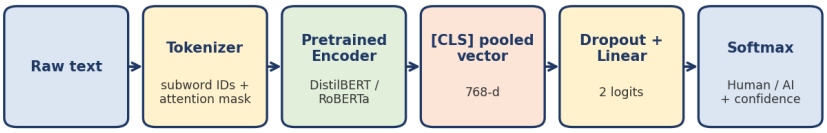


Figure 6. Transformer fine-tuning pipeline (Phase 2).

- **Contextual embeddings:** each token receives a representation that depends on its surrounding context.
- **Self-attention:** every token attends to every other token, so the model can pick up long-range stylistic and coherence patterns.
- **Classification head:** the pooled [CLS] (or first-token) representation passes through dropout and a linear layer producing two logits, followed by softmax.

5.4 Fine-Tuning

The model is trained with supervised learning on labeled human and AI-generated text, minimizing cross-entropy loss with the AdamW optimizer, a small learning rate (about 2e-5), linear warm-up and decay, and early stopping on validation F1. During fine-tuning, the model learns patterns in the training data that distinguish the two classes. The recommended progression is: TF-IDF + Logistic Regression → TF-IDF + Linear SVM → DistilBERT fine-tuning → RoBERTa fine-tuning.

6. Phase 3 — Experimental Evaluation

6.1 Experimental Setup

Dataset. The dataset is the most important part of this project. It should contain examples from both classes, ideally spanning multiple genres and topics rather than a single type of text, for example essays, news-style writing, reviews, student writing, blogs, explanations, and general question-answer pairs. For the AI class, generations from more than one model are important: a detector trained on a single model's output may learn that model's specific characteristics rather than a general human-versus-AI distinction.

Dataset splits and class balance. Data is divided into training, validation, and test sets (for example 70/15/15) with stratification on the label. Class balance is checked, and class weights or resampling are applied if needed. Duplicates are removed before splitting to prevent leakage.

Generalization test. Rather than a single random split, a more meaningful experiment trains on multiple AI sources and tests on a held-out one:

Split	Composition
Train	Human + AI Model A + AI Model B
Test	Human + AI Model C (unseen during training)

Training configuration and reproducibility. All random seeds (Python, NumPy, PyTorch) are fixed, and the same data splits are used for every model. Library versions are pinned in requirements.txt, and

hyperparameters, learning rates, batch sizes, and epochs are logged for each run so that results can be reproduced.

6.2 Ablation Study

The ablation study measures how much each feature family contributes, using the same classifier and the same splits. Results are to be completed after running the experiments.

Feature set	Accuracy	Precision	Recall	F1
Lexical only (word TF-IDF)	—	—	—	—
Lexical + char n-grams	—	—	—	—
Stylistic only (stylometric)	—	—	—	—
Semantic only (embeddings)	—	—	—	—
Hybrid (all features fused)	—	—	—	—

6.3 Performance Evaluation

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

ROC-AUC is the area under the curve of true positive rate versus false positive rate across all classification thresholds. It measures how well the classifier ranks AI texts above human texts independently of any single threshold.

Confusion matrix:

	Predicted: Human	Predicted: AI
Actual: Human	TN	FP
Actual: AI	FN	TP

Model comparison (to be completed after running the experiments):

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	—	—	—	—	—
Linear SVM	—	—	—	—	—
Random Forest	—	—	—	—	—
Hybrid fusion model	—	—	—	—	—
DistilBERT	—	—	—	—	—
RoBERTa	—	—	—	—	—

Each model is reported on both the in-distribution test set and the unseen-generator test set, followed by an error analysis of misclassified samples by length, domain, and generator.

7. Technology Stack

Layer	Tools
Data & ML baseline	pandas, numpy, scikit-learn, matplotlib, seaborn
Linguistic & semantic features	spaCy / NLTK, textstat, sentence-transformers
Deep learning	PyTorch, Hugging Face Transformers, Hugging Face Datasets, evaluate
Demo application	Streamlit

8. Streamlit Demonstration (Concept)

The final deliverable includes a lightweight web interface where a user pastes a passage of text and receives a predicted label with a confidence score.

Human vs. AI Text Detector

Enter text: [...the text to be classified...]

[Classify Text]

Prediction: AI Generated **Confidence:** 87.3%

9. Overall Project Flow

Dataset → EDA → preprocessing → stylometric, TF-IDF, and semantic feature extraction → Logistic Regression / SVM / Random Forest → evaluation → DistilBERT / RoBERTa fine-tuning → evaluation → ablation study → comparison → Streamlit demo → error and generalization analysis.